

Audix ROS 2 User Manual

This manual is for the current real-robot ROS 2 workspace. It keeps the ESP firmware unchanged and uses ROS 2 on the Raspberry Pi to talk to the ESP, read Pi GPIO sensors, control Pi GPIO outputs, and run the first ROS versions of the robot modes.

What Is Ready Now

Ready now:

- ESP UART communication using the existing `esp_correct_pid_pi.ino` protocol.
- ESP services: ping, init IMU, reset odom, reset encoders, stop, move, twist, raw command.
- IR GPIO node using the same six sensor names, pins, and baseline logic as `pi_master.py`.
- GPIO output node for buzzer, traffic LEDs, and stepper/lift.
- Basic live TF: `odom -> base_link`.
- Optional IR safety node for non-avoidance/manual modes.
- Test tools for cardinal movement and GPIO/lift checks.
- Terminal move mode.
- Terminal teleop mode using `/audix/twist`.
- Path runner mode using `/audix/move`.
- First-pass avoidance mission mode with dynamic-vs-static front obstacle logic.
- IR train mode for triggering avoidance from IR edges while standing still.

Saved for later:

- Vision.
- Full RViz robot model with URDF.

Workspace Layout

```
ros2_ws/  
  AUDIX_USER_MANUAL.md  
  AUDIX_MIGRATION_TODO.md  
  src/  
    audix_interfaces/  messages, services, actions  
    audix_robot/       ESP UART bridge  
    audix_hardware/    IR, buzzer, LEDs, stepper/lift  
    audix_bringup/     launch files  
    audix_test_tools/  simple migration test clients  
    audix_mission/     safety, move, teleop, path, IR train, avoidance
```

Copy To The Raspberry Pi

Copy or sync the full `ros2_ws` folder onto the Pi.

On the Pi:

```
cd ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

If the Pi does not have dependencies:

```
sudo apt update
sudo apt install python3-serial python3-gpiozero python3-lgpio
```

Main Launch File

Use this launch file for normal work:

```
ros2 launch audix_bringup audix.launch.py mode:=base
```

The lower-level launch still exists:

```
ros2 launch audix_bringup audix_base.launch.py
```

But the recommended file is:

```
audix_bringup audix.launch.py
```

Launch Modes

Mode	What it starts	Use
base	ESP bridge, IR node, GPIO output node	Normal bringup, manual service testing
safety	Base nodes plus IR safety node	Manual modes where any IR should stop and buzzer
ir_monitor	Base nodes plus IR edge monitor	Check IR rising/falling edges
move	Base nodes, IR safety, terminal move prompt	ROS version of simple manual MOVE mode
teleop	Base nodes, IR safety, terminal teleop prompt	Jog with /audix/twist, lift, buzzer, reset/init
path	Base nodes, IR safety, path runner	Run a waypoint list through /audix/move

Mode	What it starts	Use
avoidance	Base nodes plus avoidance mission	Goal-distance mission with dynamic/static avoidance
ir_train	Base nodes plus IR-triggered avoidance trainer	Stand still and trigger avoidance from IR
move_check	Base nodes plus cardinal movement test client	Check forward/back/right/left movement
move_check_safe	Base nodes, IR safety, movement test client	Same movement test but IR can stop robot
gpio_lift_check	Base nodes plus GPIO/lift test client	Check buzzer, LEDs, lift up/down

Common Launch Examples

Normal base bringup:

```
ros2 launch audix_bringup audix.launch.py mode:=base
```

Base bringup with IR safety:

```
ros2 launch audix_bringup audix.launch.py mode:=safety
```

Movement check at 20 cm:

```
ros2 launch audix_bringup audix.launch.py mode:=move_check move_distance_m:=0.20
```

Movement check with IR safety:

```
ros2 launch audix_bringup audix.launch.py mode:=move_check_safe  
move_distance_m:=0.20
```

GPIO/lift check:

```
ros2 launch audix_bringup audix.launch.py mode:=gpio_lift_check lift_steps:=200  
lift_speed_sps:=275.0
```

IR monitor:

```
ros2 launch audix_bringup audix.launch.py mode:=ir_monitor
```

Manual move prompt:

```
ros2 launch audix_bringup audix.launch.py mode:=move
```

At the prompt:

```
move> F 20  
move> R 15  
move> stop  
move> q
```

Terminal teleop prompt:

```
ros2 launch audix_bringup audix.launch.py mode:=teleop teleop_rpm:=25.0
```

At the prompt:

```
w/s forward/back  
a/d strafe left/right  
q/e rotate  
x stop  
u/j lift up/down  
b buzzer  
r reset odom  
i init IMU
```

Path runner:

```
ros2 launch audix_bringup audix.launch.py mode:=path  
path_waypoints:='F:20;R:20;B:10;L:20'
```

Avoidance mission:

```
ros2 launch audix_bringup audix.launch.py mode:=avoidance goal_distance_m:=1.20  
front_dynamic_hold_s:=3.0
```

IR train mode:

```
ros2 launch audix_bringup audix.launch.py mode:=ir_train
```

Run without real GPIO, useful on a laptop:

```
ros2 launch audix_bringup audix.launch.py mode:=base mock_ir:=true mock_gpio:=true
```

Use a different ESP serial port:

```
ros2 launch audix_bringup audix.launch.py mode:=base port:=/dev/ttyUSB0
```

Direct Run For Interactive Modes

If your terminal does not pass input cleanly through `ros2 launch`, keep `mode:=base` running in one terminal and run the interactive node in another:

```
ros2 run audix_mission terminal_move_node
ros2 run audix_mission terminal_teleop_node
```

This is often the smoother way to test keyboard/prompt modes.

Important Topics

Topic	Type	Meaning
<code>/audix/odom</code>	<code>nav_msgs/msg/Odometry</code>	Robot odometry from ESP telemetry
<code>/audix/esp/telemetry</code>	<code>audix_interfaces/msg/EspTelemetry</code>	Typed ESP telemetry
<code>/audix/esp/telemetry_raw</code>	<code>std_msgs/msg/String</code>	Raw ESP JSON
<code>/audix/ir/state</code>	<code>audix_interfaces/msg/IrState</code>	Six IR sensor states
<code>/tf</code>	<code>tf2_msgs/msg/TFMessage</code>	Live <code>odom</code> -> <code>base_link</code> transform
<code>/audix/mission/event</code>	<code>std_msgs/msg/String</code>	Safety/mission events

Useful checks:

```
ros2 topic echo /audix/esp/telemetry
ros2 topic echo /audix/odom
```

```
ros2 topic echo /audix/ir/state
ros2 topic echo /audix/mission/event
ros2 topic echo /tf
```

Important Services

ESP bridge:

```
ros2 service call /audix/esp/ping std_srvs/srv/Trigger {}
ros2 service call /audix/esp/init_imu std_srvs/srv/Trigger {}
ros2 service call /audix/esp/reset_odom std_srvs/srv/Trigger {}
ros2 service call /audix/esp/reset_encoders std_srvs/srv/Trigger {}
ros2 service call /audix/esp/stop std_srvs/srv/Trigger {}
```

Move:

```
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: 0.0,
distance_m: 0.20, heading_deg: 0.0, timeout_s: 10.0, wait_for_done: true}"
```

Direction mapping:

Angle	Robot movement
0	forward
180	backward
-90	right
90	left

Buzzer:

```
ros2 service call /audix/gpio/set_buzzer std_srvs/srv/SetBool "{data: true}"
ros2 service call /audix/gpio/set_buzzer std_srvs/srv/SetBool "{data: false}"
```

Traffic LEDs:

```
ros2 service call /audix/gpio/set_traffic_light
audix_interfaces/srv/SetTrafficLight "{red: true, yellow: false, green: false}"
ros2 service call /audix/gpio/set_traffic_light
audix_interfaces/srv/SetTrafficLight "{red: false, yellow: true, green: false}"
ros2 service call /audix/gpio/set_traffic_light
audix_interfaces/srv/SetTrafficLight "{red: false, yellow: false, green: true}"
```

```
ros2 service call /audix/gpio/set_traffic_light
audix_interfaces/srv/SetTrafficLight "{red: false, yellow: false, green: false}"
```

Lift:

```
ros2 service call /audix/lift/move_steps audix_interfaces/srv/LiftMoveSteps "
{steps: 200, direction: 1, speed_sps: 275.0}"
ros2 service call /audix/lift/move_steps audix_interfaces/srv/LiftMoveSteps "
{steps: 200, direction: -1, speed_sps: 275.0}"
```

Lift direction:

Direction	Meaning
1	up
-1	down

Suggested Test Order

1. Build the workspace.
2. Launch `mode:=base`.
3. Test `/audix/esp/ping`.
4. Echo `/audix/esp/telemetry`.
5. Echo `/audix/odom`.
6. Test `/audix/move` manually at a short distance.
7. Echo `/audix/ir/state` and block each sensor by hand.
8. Test buzzer and traffic LEDs.
9. Test lift up/down.
10. Try `mode:=move_check`.
11. Try `mode:=safety` or `mode:=move_check_safe`.
12. Try `mode:=move`.
13. Try `mode:=teleop`.
14. Try `mode:=path` with a short waypoint list.
15. Try `mode:=ir_train`.
16. Try `mode:=avoidance` with a short goal distance first.

Do not start path/avoidance until base movement, IR, GPIO, lift, and teleop pass.

Notes About RViz And URDF

The bridge now publishes a basic live transform:

```
odom -> base_link
```

That is enough to begin RViz odometry visualization.

For a real robot model in RViz, we will later add your URDF/CAD-derived description. The important point is that RViz should display live real odometry from `/audix/odom` and `/tf`, not simulated motion.